

数据库原理与应用

第4章 SQL语言基础及 数据定义功能



第4章 SQL 语言基础及数据定义功能



4.1 SQL 语言概述



4.2 SQL 支持的数据类型



4.3 数据定义功能

4.1 SQL 语言概述

- **4.1.1 SQL 语言的发展**
- **4.1.2 SQL 语言的特点**
- **4.1.3 SQL 语言功能概述**

4.1.1 SQL 语言的发展

- **1986 年 10 月**由美国 **ANSI** 公布最早的 **SQL** 标准。
- **1989 年 4 月**，**ISO** 提出了具备完整性特征的 **SQL**，称为 **SQL-89**。
- **1992 年 11 月**，**ISO** 又公布了新的 **SQL** 标准，称为 **SQL-92**（以上均为关系形式）。
- **1999 年**颁布 **SQL-99**，是 **SQL92** 的扩展。

4.1.2 SQL 语言的特点

- **1. 一体化**
- **2. 高度非过程化**
- **3. 简洁**
- **4. 使用方式多样**

4. 1. 3 SQL 语言功能概述

四部分：数据定义功能、数据控制功能、数据查询功能和数据操纵功能。

SQL 功 能	命令动词
数据查询	SELECT
数据定义	CREATE 、 DROP 、 ALTER
数据操纵	INSERT 、 UPDATE 、 DELETE
数据控制	GRANT 、 REVOKE 、 DENY

4.2 SQL 支持的数据类型

- 数值型
- 字符串型
- 日期时间型

数值型

□ 准确型

■ 整数

Bigint: 8 字节,

Int: 4 字节

Smallint: 2 字节,
(0~255)

Tinyint: 1 字节

Bit: 1 位, 存储 1 或 0

■ 小数

Numeric (p,q) 或 Decimal (p,q),

其中: **p** 为数字位长度, **q**: 小数位长度。

□ 近似型

Float: 8 字节

Real: 4 字节

字符串型

- 普通编码字符串类型
- 统一字符编码字符串类型
- 二进制字符串类型

普通编码字符串类型 (ASCII 代码体系)

- **Char(n)**: 定长存储, $n \leq 8000$
- **Varchar(n)**: 不定长存储 (按实际长度存储), 长度最大不超过 n , $n \leq 8000$

注: n 为字节个数, , 一个英文字符占一个字节, 但是 **1** 个汉字字符占 **2** 个字节!!!

- **Text**: 可以存储大于 **8000** 字节的文本
- **Varchar(max)**: 存储大于 **8000** 字节



统一字符编码字符串类型 (Unicode 代码体系)

- **nchar (n)** : 定长存储, **$n \leq 4000$**
- **nvarchar (n)** : 不定长存储, 长度最大不超过 **n** , **$n \leq 4000$**
- **ntext** : 存储大于 **8000** 字节的文本
- **Nvarchar(max)** : 存储大于 **8000** 字节的文本
- 以上均为 **Unicode** 代码: 其中的 **n** 为字符数, 每个字符 (无论什么字符) 均为两个字节长度。

二进制字符串类型

- ❑ **Binary(n)** : 固定长度, $n \leq 8000$
- ❑ **Varbinary(n)** : 可变长度, $n \leq 8000$
注: **n** 为二进制数据的字节数
- ❑ **image** : 用于存储超过 8000 字节的大二进制文件 (将被淘汰的类型)
- ❑ **Varbinary(max)** : 用于存储超过 8000 字节的大二进制文件。

日期时间类型

□ **Datetime** : 8 字节, 年月日时分秒毫

秒
(例: ‘ 2001/08/03
10:30:00.000’)

存储从 **1753 年 1 月 1 日到 9999 年
12 月 31 日** 的日期和时间数据

□ **SmallDateTime** : 4 字节, 年月日时分

(例: ‘ 2001/08/03

10:30:00’)

日期时间类型

☐ **Date:**

☐ **Time:**

4.3 数据定义功能

- 4.3.1 架构定义与删除
- 4.3.2 基本表

SQL 数据定义功能

对 象	创 建	修 改	删 除
架构	CREATE SCHEMA		DROP SCHEMA
表	CREATE TABLE	ALTER TABLE	DROP TABLE
视图	CREATE VIEW	ALTER VIEW	DROP VIEW
索引	CREATE INDEX	ALTER INDEX	DROP INDEX

架构基本概念

- **架构**（**schema**，也称为**模式**）是数据库下的一个逻辑命名空间，
- 可以存放表、视图等数据库对象，
- 是一个数据库对象的容器。
- 如果将数据库比喻为操作系统，则架构就相当于操作系统中的目录，架构中的对象就相当于目录下的文件。

架构基本概念

- 一个数据库可以包含一个或多个架构，由特定的授权用户名所拥有。
- 在同一个数据库中，架构的名字必须是唯一的。
- 属于一个架构的对象称为**架构对象**。架构对象可以是：基本表、视图、触发器等。

定义架构

CREATE SCHEMA [<架构名 >]

AUTHORIZATION <用户名 >

- 如果没有指定 <架构名 >，则 <架构名 > 隐含为 <用户名 >。
- 一个 <用户名 > 可以拥有多个架构。
- 执行创建架构语句的用户必须具有管理员权限，或 CREATE SCHEMA 权限。

示例

- 例 1 . 为用户 “ZHANG” 定义一个架构，架构名为 “S_C” 。

```
CREATE SCHEMA S_C AUTHORIZATION ZHANG
```

- 例 2 . 定义一个用隐含名字的架构。

```
CREATE SCHEMA AUTHORIZATION ZHANG
```

示例

□ 例 3 . 在定义架构的同时定义表。

```
CREATE SCHEMA TEST AUTHORIZATION ZHANG  
CREATE TABLE T1(  
    C1 INT,  
    C2 CHAR(10),  
    C3 SMALLDATETIME,  
    C4 NUMERIC(4, 1)  
)
```

删除架构

DROP SCHEMA <架构名>

{ <CASCADE> | <RESTRICT> }

- CASCADE**：删除架构的同时将该架构中所有的对象一起删除。
- RESTRICT**：如果被删除的架构中包含对象，则拒绝删除此架构。

说明

- ❑ 不同 DBMS 的 DROP SCHEMA 语句的语法格式和执行略有不同。
- ❑ SQL Server 2008 的 DROP SCHEMA 语句没有可选项，其语法格式为：
DROP SCHEMA <架构名>
- ❑ 在 SQL Server 2008 中只能删除不包含任何对象的架构。

定义基本表

```
CREATE TABLE < 表名 > (  
    < 列名 > < 数据类型 > [ 列级完整性约束 ]  
    {, < 列名 > < 数据类型 >  
        [ 列级完整性约束 ] ... }  
    [, 表级完整性约束 ] )
```

在列级完整性约束定义处可定义的约束

- ❑ **NOT NULL** : 限制列取值非空。
- ❑ **DEFAULT** : 给定列的默认值。
- ❑ **UNIQUE** : 限制列取值不重。
- ❑ **CHECK** : 限制列的取值范围。
- ❑ **PRIMARY KEY** : 指定本列为主键。
- ❑ **FOREIGN KEY** : 定义本列为引用其他表的外键。

几点说明

- **NOT NULL** 和 **DEFAULT** 只能是列级完整性约束；
- 其他约束均可在表级完整性约束处定义。
- 注意以下几点：
 - 如果 **CHECK** 约束是定义多列之间的取值约束，则只能在表级完整性约束处定义；
 - 如果在表级完整性约束处定义主键和唯一值约束，则应将主键列和唯一值约束列用圆括号括起来，如：**PRIMARY KEY (列 1 { [, 列 2]**

... })

- 本章只介绍非空约束、主键约束和外键约束。

非空约束

□ **< 列名 > < 类型 > NOT NULL**

例: **sname char(10) NOT NULL**

主键约束

PRIMARY KEY [(< 列名 > [, ... n])]

- 如果在列级完整性约束处定义单列主键，则可省略括号。

例 1 : SNO char(7) PRIMARY KEY

**例 2 : PRIMARY KEY(SNO)
PRIMARY KEY(SNO , CNO)**

外键约束

[FOREIGN KEY (<外键列名 >)]

REFERENCES <表名 > (<主键列名 >)

- 如果是在列级完整性约束处，则可省略
“ FOREIGN KEY (<列名 >) ” 部分。

例：定义 SC表的 Sno 外键。

FOREIGN KEY(Sno)

REFERENCES Student (Sno)

唯一值约束

UN Q UE [(<列名> [, ... n])]

- 如果在列级完整性约束处定义单列的唯一值约束，则可省略方括号中的内容。
- 用于限制一个列的取值不重复，或者是多个列的组合取值不重复。
 - 有 UN Q UE 约束的列允许有一个空值；
 - 在一个表中可以定义多个 UN Q UE 约束
 - 可以在一个列或多个列上定义 UN Q UE 约束

默认值约束

- 用于提供列的默认值。当在表中插入数据时，若没有为有默认值约束的列提供值，则系统自动使用默认值。
- 一个默认值约束只能为一个列提供默认值，且默认值约束必须是列级约束

默认值约束 (续)

- 默认值约束的定义有两种形式:
 - 在创建表时定义 DEFAULT 约束
DEFAULT 常量表达式
 - 为已创建好的表添加 Default 约束
DEFAULT 常量表达式 FOR 列名

限制列取值约束

CHECK(逻辑表达式)

- 用于限制列的取值在指定范围内,
- 例如, 人的性别只能是 { 男, 女 }
- CHECK所限制的列必须在同一个表中
- 如果 CHECK约束是定义多列之间的取值约束, 则只能在“表级完整性约束定义”处定义。

示例

Jobs 表

列 名	含 义	数 据 类 型	约 束
Jid	工作编号	char(6)	主键
Descp	工作描述	nvarchar(20)	非空
EduReq	学历要求	nchar(6)	默认值：本科
MinSalary	最低工资	int	
MaxSalary	最高工作	int	大于等于最低工资

创建 Jobs 表

```
CREATE TABLE Jobs (  
    Jid          char(6)          PRIMARY KEY,  
    Descp        nchar(20)        NOT NULL,  
    EduReq        nchar(6)        DEFAULT '本科',  
    MinSalary    int              ,  
    MaxSalary    int              ,  
    CHECK( MaxSalary >= MinSalary )  
)
```

示例

Employees 表

列 名	含 义	数 据 类 型	约 束
Eid	职工号	char(10)	主键
Ename	姓名	nchar(6)	非空
Sex	性别	nchar(1)	取值范围：“男”或“女”
BrithDate	出生日期	date	
JobDate	参加工作日期	dateTime	默认为系统当前日期时间
Sid	身份证号	char(18)	取值不重
Jid	所干工作编号	char(6)	外键，引用工作表的工作编号
Tel	联系电话	char(11)	

创建 Employees 表

```
CREATE TABLE Employees (  
    Eid char(10) ,  
    Ename nvarchar(20) NOT NULL,  
    Sex nchar(1) CHECK( Sex = '男' OR Sex =  
'女'),  
    BirthDate date ,  
    JobDate datetime DEFAULT GetDate(),  
    Sid char(18) UNIQUE ,  
    Jid char(6) ,  
    Tel char(11) ,  
    PRIMARY KEY(Eid) ,  
    FOREIGN KEY(Jid) REFERENCES Jobs(Jid) )
```

修改表结构

- ❑ 在定义完表之后，如果需求有变化，比如添加列、删除列或修改列定义，可以使用 **ALTER TABLE** 语句实现。
- ❑ **ALTER TABLE** 语句可以对表添加列、删除列、修改列的定义、定义主键、外键，也可以添加和删除约束。

修改表结构

ALTER TABLE < 表名 >

[ALTER COLUMN < 列名 > < 新数据类型 >]

| [ADD < 列名 > < 数据类型 > [约束]]

| [DROP COLUMN < 列名 >]

| [ADD [constraint < 约束名 >] 约束定义]

| [DROP [constraint] < 约束名 >]

示例

- ❑ 例 1 . 为 **Employees** 表添加工资列，此列的列名为 **Salary** ，数据类型为 **int** ，允许空。

```
ALTER TABLE Employees  
ADD Salary INT
```

示例

- 例 2. 将 **Jobs** 表的 **Descp** 列的数据类型改为 **NCHAR(40)**。

ALTER TABLE Jobs

ALTER COLUMN Descp NCHAR(40)

- 例 3. 删除 **Employees** 表的 **Tel** 列。

ALTER TABLE Employees

DROP COLUMN Tel

示例

□ 例 4. 为 Jobs 表中 MinSalary 列添加约束：大于等于 1600。

ALTER TABLE Jobs

ADD CHECK(MinSalary >= 1600)

删除表

DROP TABLE < 表名 >

{ [, < 表名 >] ... }

□ 例 4. 删除 **Employees** 表的语句为:

DROP TABLE Employees